How the Video Trajectory Learner Works

A plain-language guide, followed by the technical details

Video Representation Learning

October 2026

Contents

1 The short version **2**

2 Begin with a video and a sequence of summaries **2**
2.1 An important distinction about "what happened at 1.0 seconds" 2

3 The trajectory model: a small learner after the encoder **2**
3.1 A gentle analogy 3

4 Future prediction without looking ahead **3**
4.1 Timeline example 3
4.2 What a forecast looks like 4
4.3 What is the persistence comparison? 4

5 Four training goals (the four loss terms) **4**
5.1 A. Semantic loss: separate clean and spill summaries 4
5.1.1 Small example 4
5.1.2 What the curriculum does 5
5.1.3 What this loss does not prove 5
5.2 B. Alignment loss: matched views should tell a consistent temporal story 5
5.3 C. Prediction loss: forecast later frozen-encoder features 5
5.4 D. Retention loss: do not drift too far from the encoder representation 5

6 How the four goals become one training signal **6**

7 Technical detail: the exact ViCo semantic competition **6**

8 Technical detail: the other terms and the total **7**

9 What happens in one training update? **8**

10 A little more technical: what the tensors mean **8**

11 Training setup and interpretation boundaries **8**

12 Glossary **9**

13 Where this explanation comes from **9**

1 The short version

This project uses one large, pretrained video encoder and a smaller trainable trajectory model.

The encoder turns each *video prefix*—everything seen from the start through a particular time—into a list of numbers called a feature. It is kept frozen. The trajectory model reads the sequence of these features and learns three things:

1. **Summarize the whole video for its label.** Clean videos should have summaries distinct from spill videos.
2. **Describe how the video representation changes over time.** Two matched views of the same video should have compatible dynamics.
3. **Predict later encoder features from earlier context.** Its forecast is compared with the frozen encoder's feature for a later video prefix.

The training loss combines these goals. It does not update the large encoder; it updates the smaller trajectory model and its heads.

> **One encoder, two roles in training:** The frozen encoder produces features that are both inputs and reference targets. When a later feature is used as the answer against which a forecast is scored, it is sometimes called a "teacher feature." That word describes its role; it does not mean there is a second encoder.

2 Begin with a video and a sequence of summaries

Imagine a camera records a scene. Every half-second, the encoder is asked to summarize the video seen so far:

```
z1 = encoder(video from start through 0.5 s)
z2 = encoder(video from start through 1.0 s)
z3 = encoder(video from start through 1.5 s)
z4 = encoder(video from start through 2.0 s)
...
```

Each `z` is a vector of numbers. It is not a sentence, image, class label, or single-frame feature. It is the encoder's representation of a prefix of the video.

The extraction script makes each prefix separately. This matters because the encoder can look across all frames it is given. If the script encoded the full video once and then reused the early outputs, those outputs could have already been influenced by frames from later in the video. Encoding each prefix separately prevents that future-frame leakage.

The vectors are saved in a cache. Training the trajectory model then uses that cache; it does not rerun or fine-tune the encoder.

2.1 An important distinction about "what happened at 1.0 seconds"

The feature `z2` is not necessarily "what happened in the frame at 1.0 s." It represents the **whole prefix through 1.0 s**. It can include information from all the frames the encoder saw in that prefix.

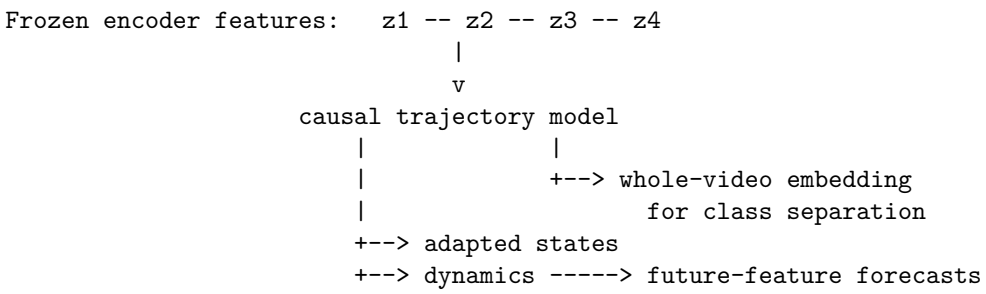
For example, if the first evidence of a spill appears at 0.8 s, `z2` may represent that evidence. `z1`, made from the prefix ending at 0.5 s, could not represent evidence that had not appeared yet.

3 The trajectory model: a small learner after the encoder

The trajectory model accepts a sequence such as `z1, z2, z3, z4`. It learns how to adapt those features over time. For each position, it produces:

- **State:** an adjusted version of the encoder feature at that position.
- **Dynamics:** an internal representation used to help forecast later features.
- **Whole-video embedding:** one summary vector for the complete observed sequence, used for the clean/spill learning objective.

A rough picture:



"Causal" here means that the model's result at a time can use the current and earlier features, but not later features:

```
state at z1 can use: z1
state at z2 can use: z1, z2
state at z3 can use: z1, z2, z3
```

This does not mean the model forgets earlier observations at each step. The state at `z3` can use the whole prefix of features `z1, z2, z3`.

3.1 A gentle analogy

Imagine that the encoder writes a running situation summary as it watches a security-camera video. A second, smaller learner reads those summaries and learns:

- what final summaries tend to distinguish clean scenes from spill scenes;
- which changes in summaries tend to occur as a video progresses; and
- what a later summary might look like, given the summaries already seen.

The learner is not predicting the future pixels. It predicts a future **encoder summary**. It cannot know an unpredictable event before there is evidence for it. It can only learn patterns that the training examples make predictable from the past.

4 Future prediction without looking ahead

This is the part most easily misunderstood. Keep two roles separate:

1. **Input to a forecast:** information the model can use when making its guess.
2. **Target for scoring the forecast:** the later feature that tells the training code how good the guess was.

The target is allowed to be read **after the forecast has been formed** to calculate an error. It is not an input used to make that forecast. This is similar to a practice problem: a student makes a guess using the clues available, then compares it with the answer key to learn from the mistake.

4.1 Timeline example

Assume, just for illustration, that adjacent feature steps are 0.5 seconds apart. At the position `z2`, which ends at 1.0 s:

```
Observed context at this position:
    z1 (prefix through 0.5 s), z2 (prefix through 1.0 s)

Separate forecasts made from the context at z2:
    horizon 1: predict z3 (prefix through 1.5 s)
    horizon 2: predict z4 (prefix through 2.0 s)
    horizon 4: predict z6 (prefix through 3.0 s)
```

Those are three forecasts from the **same current position**. They are not a chain of guesses. In particular, the horizon-2 forecast of `z4` does not use the model's horizon-1 guess of `z3`.

The model also forecasts from other valid positions. At position `z3`, it has the context `z1, z2, z3` and can make its own one-step forecast of `z4`. Therefore, the same future feature `z4` may serve as a target for forecasts made at multiple earlier positions/horizons.

```
Forecast from z2: context z1,z2       -----> target z4 (horizon 2)
Forecast from z3: context z1,z2,z3    -----> target z4 (horizon 1)
```

The code constructs all valid earlier/later pairs for each configured horizon. It uses actual timestamps to tell the prediction head how much time elapsed. The approximate durations depend on the sampled video cadence; the horizon number itself is a count of feature steps.

4.2 What a forecast looks like

Conceptually, at position `t` the model starts with its adapted state and adds a learned estimate of how the representation may change:

```
forecast = current adapted state
         + predicted change(dynamics at t, elapsed time)
```

For example, using tiny made-up three-number vectors:

```
current adapted state:  [ 0.2, -0.4,  1.1]
predicted change:       [ 0.1,  0.3, -0.2]
forecast:               [ 0.3, -0.1,  0.9]
later encoder target:   [ 0.4, -0.2,  0.8]
```

The prediction error measures the difference between the forecast and target. These numbers are illustrative only; real features are much wider vectors.

4.3 What is the persistence comparison?

A very simple forecast is: "The feature will stay as it is now." The code measures this **persistence baseline** too. It compares the current frozen encoder feature directly with the future frozen encoder feature.

- If the learned forecast has lower error than persistence, it beat "assume nothing changes" on those evaluated pairs.
- If its error is equal to persistence, it did not improve on that simple baseline.
- This comparison is not an accuracy percentage and does not prove generalization to unseen videos.

5 Four training goals (the four loss terms)

A *loss* is a numerical penalty for a behavior the training process wants to improve. The model gets penalties for several different behaviors, because this project wants more than a class label alone.

5.1 A. Semantic loss: separate clean and spill summaries

The trajectory model combines valid states into one embedding per video view. The semantic loss compares these embeddings using the class labels:

- **positive pair:** two different embeddings whose labels match;
- **negative pair:** two embeddings whose labels differ.

The goal is for same-class summaries to compete successfully against different-class summaries. The labels used by this project are `clean = 0` and `cold/warm spill = 1`.

5.1.1 Small example

Suppose the training batch includes views of:

```
Video A: clean
Video B: clean
```

`Video C: spill`
`Video D: spill`

If Video A's view is the anchor:

- Video B's view is a positive because both are clean.
- Views of Videos C and D are negatives because they are spill.
- The other view of Video A is also a same-class positive; it has the same source identity and receives weight one.

The loss asks each positive to hold up against the anchor's negative evidence. It does not use one shared denominator across all positives as standard shared-denominator SupCon would. Each positive has its own competition against the negatives. The implementation is the ViCo-LWIR Eq. 10 objective in `src/vision_model/training/vico.py`.

5.1.2 What the curriculum does

The positive-pair weights can change during the configured curriculum. The implementation computes uncertainty/rank-based weights as detached selection statistics. In everyday terms, the loss can gradually adjust which same-class relationships receive extra emphasis. The model is not allowed to reduce the loss by manipulating the ranking calculation itself.

The negative weights also emphasize negative evidence according to the implemented ViCo weighting rule, including its square-root factor based on negative count. The semantic calculation is done in float64 to preserve small loss values more accurately.

5.1.3 What this loss does not prove

A low semantic loss shows fit to the supplied labels. It does **not** by itself prove that the model recognizes the physical liquid, localizes the spill in pixels, or generalizes to new scenes.

5.2 B. Alignment loss: matched views should tell a consistent temporal story

During training, the same source video is represented in two synchronized views. At each valid time step, the model compares the views' dynamics vectors using Smooth L1 error.

In plain language: a permitted view change (such as a whole-clip horizontal mirror) should not make the model describe the same video's temporal behavior as if it were a different process.

This is time-by-time matching. It is not a rule that adjacent moments must be similar, and it is not a direct "make the video smooth" penalty.

5.3 C. Prediction loss: forecast later frozen-encoder features

For each valid time and each configured horizon:

1. The causal model provides the current adapted state and dynamics.
2. The future head receives the dynamics and actual timestamp difference.
3. It predicts a residual change and adds it to the current adapted state.
4. The result is compared with the normalized frozen-encoder feature at the later prefix using Smooth L1.

Prediction errors are summed across the valid pairs and horizons, then divided by the number of valid scalar target values. Padded positions are excluded. This is the objective that teaches the adapter to model predictable changes in the encoder's representation.

It is not a future-frame reconstruction loss, a future-class loss, or a spill-mask loss.

5.4 D. Retention loss: do not drift too far from the encoder representation

At every valid time step, the model compares its normalized adapted state with the normalized frozen encoder feature. Smooth L1 penalizes differences.

In plain language: the adapter is allowed to refine the representation, but this term discourages it from throwing away the information already present in the frozen encoder features.

The adapter's residual projection starts at zero, so initially its state matches its input feature. Consequently, retention loss starts at zero before the adapter learns to alter the states.

6 How the four goals become one training signal

The default total is:

```
total = semantic
      + 0.1 × alignment
      + 1.0 × prediction
      + 0.1 × retention
```

Think of each loss as a separate kind of problem on the model's report card. The multipliers scale how much each measured error contributes to the combined score. For example, if the measured prediction loss is `0.08` and its weight is `1.0`, it contributes `0.08`. If its weight were `0.5`, it would contribute `0.04`. Changing the weight does not itself change the prediction; it changes the contribution of the prediction error to the total.

Here is a made-up example:

Goal	Measured loss	Weight	Added amount
Class separation	0.20	1.0	0.20
View alignment	0.40	0.1	0.04
Future prediction	0.08	1.0	0.08
Feature retention	0.30	0.1	0.03
Total			**0.35**

These are example values, not measured results. The training optimizer uses the total to adjust the trainable trajectory model. The individual terms are also reported so a lower total can be understood: one term may improve while another gets worse.

A weight of `1.0` does not guarantee that a term has the largest effect on learning. Loss magnitudes and their gradients also matter. The persistence baseline is reported for comparison but is not added to the total.

7 Technical detail: the exact ViCo semantic competition

For an anchor video embedding `i`, let `P_i` be its positive embeddings and `N_i` its negative embeddings. Let `s_ij` be the cosine similarity between normalized embeddings `i` and `j`. Higher cosine means the vectors point in more similar directions. The temperature `tau` controls how strongly similarity differences affect the competition; this implementation uses `tau = 0.1`.

For each individual positive `j`, the implemented pair loss is:

$$\ell_{ij} = \text{softplus}\left(\log\left[\epsilon + \sum_{k \in N_i} \alpha_{ik} \exp(s_{ik}/\tau)\right] - \left[s_{ij}/\tau + \log(w_{ij})\right]\right).$$

In words:

- The first part measures the anchor's weighted negative evidence.
- The second part measures the selected positive's evidence, including its positive weight.
- `softplus` turns their difference into a smooth nonnegative penalty.
- If a positive is strong relative to the negatives, its penalty becomes small. If negative evidence dominates, the penalty becomes larger.

Each positive gets its **own** comparison against the anchor's negatives. The code averages positive-pair penalties within an anchor, then averages over anchors:

$$L_{\text{ViCo}} = \frac{1}{B} \sum_i \frac{1}{|P_i|} \sum_{j \in P_i} \ell_{ij}.$$

This order of averaging matters. It is not one shared softmax denominator across all positives, and it does not add a `log(number of positives)` floor.

The weights have specific roles:

- `alpha_ik` weights negative examples according to the implemented ViCo negative weighting rule, including the square-root negative-count factor.
- `w_ij` weights positive pairs according to an uncertainty-rank curriculum. Same-source views keep weight one.
- Both are detached: they affect which pairs count more, but the model cannot improve the objective by changing the ranking calculation itself.
- `epsilon = 1e-6` provides the small stabilizing term shown in the equation.

The implementation calculates this small semantic loss in float64. It reports additional diagnostics such as worst positive-pair loss, minimum same-class cosine, maximum cross-class cosine, and minimum per-anchor separation. These diagnostics help reveal whether a low average hides a difficult pair.

8 Technical detail: the other terms and the total

Let `m_it` indicate whether time position `t` for sequence `i` is valid. Let `d_it` be its dynamics vector, `h_it` its adapted state, and `z_it` the frozen encoder feature. `LN` means layer normalization across the feature dimension. `SL1` means Smooth L1.

Alignment compares the two matched views of each source at the same valid times:

$$L_{\text{align}} = \text{SL1}(d_{it}^{(view\,1)}, d_{it}^{(view\,2)}) \quad \text{over all valid } (i,t).$$

Retention compares each valid adapted state with its frozen feature after both are normalized:

$$L_{\text{keep}} = \text{SL1}(LN(h_{it}), LN(z_{it})) \quad \text{over valid } (i,t).$$

Prediction makes a set of valid forecasts. For a position `t` and horizon `k`, the actual elapsed time is `delta_t = timestamps[t+k] - timestamps[t]`. The forecast is:

$$\hat{z}_{t+k} = LN(h_t) + g(d_t, delta_t),$$

where `g` is the learned future-change head. The target is the normalized encoder feature at position `t+k`, detached from gradient updates. The code sums Smooth L1 errors across all valid positions and configured horizons, then divides by the total number of valid scalar feature targets:

$$L_{\text{predict}} = \frac{\sum_{\text{valid } (i,t,k)} \text{SL1}_{\text{sum}}(\hat{z}_{i,t+k}, LN(z_{i,t+k}))}{\text{number of valid scalar targets}}.$$

Consequently, horizons contribute according to the number of valid pairs they provide; the implementation does not first average each horizon separately and then give every horizon equal weight. The persistence metric performs the same target comparison using the current frozen feature as the forecast. It is a baseline, not a fifth loss term.

Finally, the reported training objective is:

$$L_{\text{total}} = L_{\text{ViCo}} + 0.1 L_{\text{align}} + 1.0 L_{\text{predict}} + 0.1 L_{\text{keep}}$$

The optimizer differentiates this sum with respect to the trainable trajectory model. A lower term is not automatically an improvement in every scientific claim: each term measures only the behavior it was designed to measure.

9 What happens in one training update?

The default training script performs one full-batch optimizer update per epoch. At a high level:

1. Load the cached features, timestamps, validity masks, and labels.
2. Choose two views for every source video.
3. Run the trajectory model on the sequences.
4. Calculate the ViCo semantic, alignment, prediction, and retention terms.
5. Combine them into `total`.
6. Backpropagate the total through the trainable trajectory model.
7. Clip the gradient norm and update its parameters with AdamW.
8. Evaluate and record the separate losses and diagnostics.

The encoder is frozen and cached, so step 6 does not train the encoder.

10 A little more technical: what the tensors mean

The training loss receives two views per source:

```
teacher:    [number of videos, 2 views, time steps, feature width]
valid:      [number of videos, time steps]
timestamps: [number of videos, time steps]
labels:     [number of videos]
```

The two views are flattened to the model batch dimension. Thus each video contributes two sequence embeddings to ViCo. Its label is repeated for both, and both embeddings share a source ID so the loss recognizes them as views of the same source.

The model returns:

```
states:    [videos × views, time, feature width]
dynamics:  [videos × views, time, hidden width]
embedding: [videos × views, semantic embedding width]
```

The model standardizes feature vectors with layer normalization. Its Transformer uses a strict upper-triangular attention mask, so position `t` cannot attend to positions later than `t`. A key-padding mask also excludes invalid padded positions.

For a horizon `k`, valid prediction pairs are constructed from positions `t` and `t+k`. The forecast is:

```
predicted = standardized_state[t] + future_delta(
    concatenate(dynamics[t], timestamp[t+k] - timestamp[t])
)
```

The target is the detached, standardized frozen feature at `t+k`. The teacher feature is detached, and prefix extraction is causal, so the training target does not update the encoder and the earlier feature was not computed with future frames.

Smooth L1 behaves quadratically for small differences and approximately linearly for large differences. It therefore penalizes errors without letting one unusually large component dominate as strongly as squared error can.

11 Training setup and interpretation boundaries

The script defaults to 100 epochs, seed 239, AdamW learning rate `1e-3`, no weight decay, epsilon `1e-12`, and gradient-norm clipping at 5. One epoch is one full-batch optimizer update. AdamW state is reset once immediately after the curriculum, before update 21 with the default 20-epoch curriculum.

Evaluation curves use fixed final-curriculum pair weights at every recorded epoch so epoch-to-epoch values are comparable, even though training pair weights change during the curriculum.

Keep these conclusions separate:

- Low semantic loss: the model fit the supplied class labels.
- Prediction below persistence: the model beat the unchanged-feature baseline on the evaluated pairs.
- Localization: requires spatial evidence and validation against real masks using metrics such as IoU or pointing accuracy, preferably on unseen scenes.
- Generalization: requires videos or scene groups excluded from every training operation, including representation fitting and prototype construction.

The current feature adapter receives spatially averaged tokens. It can select among times, but it cannot produce a learned patch-level spill mask from these inputs. A good class score or attention plot is not evidence of pixel-level localization.

12 Glossary

Term	Everyday meaning
Encoder	The pretrained video feature-maker.
Frozen	Its parameters are not changed during this training stage.
Feature / embedding	A list of numbers representing a video or video prefix.
Prefix	The video from its start through a chosen time.
Teacher feature	A frozen encoder feature used as a training target; not a second encoder.
State	The trajectory model's adjusted feature at one time.
Dynamics	A learned time-dependent representation used to help forecast.
Horizon	How many feature steps ahead a forecast target is.
Positive / negative	Same-label / different-label pair for the semantic objective.
Loss	A numerical penalty; training tries to reduce its weighted combination.
Padding	Repeated or placeholder positions used to make sequence lengths match; excluded from valid losses.
Persistence	A baseline forecast that assumes the current encoder feature does not change.

13 Where this explanation comes from

This guide follows the current implementation in:

- `scripts/extract_causal_video_features.py` — separately encodes each valid prefix and caches its frozen feature.
- `src/vision_model/training/trajectory.py` — defines `TrajectoryModel` and combines the four loss terms in `trajectory_loss`.
- `src/vision_model/training/vico.py` — implements the pairwise ViCo-LWIR semantic objective and its weighting rules.
- `scripts/train_trajectory.py` — selects views, performs optimizer updates, and records metrics.

The code is authoritative if this explanatory guide and implementation ever differ.